Università di Genova

UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MOBILE ROBOTS

Localization Lab

Author:

Beaini Charbel

Student ID:

S8027823

Professors:

Gaetan Garcia

May 21, 2026

You have done a good job. It looks like you understand the EKF well now. Not often do I see a report that reflects this much overall understanding. Hopefully, you learned in the process.
Maybe have a quick look at the diagonal45degrees case again with and without correct initial theta, and make sure you can spot that something is wrong when theta is initialized at 0.

Contents

1 Question 5

1.1 The state X

$$X = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \tag{1}$$

where:

- x = robot position in world frame in mm
- y = robot position in world frame in mm
- θ = robot orientation in radians

OK

1.2 The input U

In `PlotRawData.m`:

$$\Delta q = \begin{bmatrix} q_R(i) - q_R(i-1) \\ q_L(i) - q_L(i-1) \end{bmatrix} \tag{2}$$

$$U(:, i) = \text{jointToCartesian} \cdot \Delta q \tag{3}$$

And in `RobotAndSensorDefinition.m`:

$$\text{jointToCartesian} = \begin{bmatrix} \frac{r_{\text{wheel}}}{2} & \frac{r_{\text{wheel}}}{2} \\ \frac{r_{\text{wheel}}}{\text{trackGauge}} & -\frac{r_{\text{wheel}}}{\text{trackGauge}} \end{bmatrix} \tag{4}$$

Therefore:

$$U = \begin{bmatrix} \Delta S \\ \Delta \theta \end{bmatrix} \tag{5}$$

where:

- ΔS = forward displacement during one sample in mm It is signed. So it may be backwards if negative.
- $\Delta \theta$ = orientation change during one sample in radians

SBased on this, we can deduce that U is not speed. The speed is only computed later for plotting:

$$\frac{U(1, :)}{\text{samplingPeriod}} \tag{6}$$

Note: The comment is misleading at line 47 "joint speed to Cartesian speed". It is not speed yet, which is also confirmed by the code since we divide by the sampling time at line 67 to plot the speed. Noted.

1.3 The evolution model

The evolution model is therefore:

$$x_{\text{next}} = x + \Delta S \cos(\theta) \tag{7}$$

$$y_{\text{next}} = y + \Delta S \sin(\theta) \tag{8}$$

$$\theta_{\text{next}} = \theta + \Delta \theta \tag{9}$$

2 Question 6

2.1 Jacobians of the Evolution Model

Matrix A: Jacobian of the evolution model with respect to the state X.

$$A = \frac{\partial f}{\partial X} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} & \frac{\partial f_1}{\partial \theta} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} & \frac{\partial f_2}{\partial \theta} \\ \frac{\partial f_3}{\partial x} & \frac{\partial f_3}{\partial y} & \frac{\partial f_3}{\partial \theta} \end{bmatrix} \tag{10}$$

$$A = \begin{bmatrix} 1 & 0 & -U(1)\sin(X(3)) \\ 0 & 1 & U(1)\cos(X(3)) \\ 0 & 0 & 1 \end{bmatrix} \quad \text{OK} \tag{11}$$

Matrix B: Jacobian of the evolution model with respect to the input U.

$$B = \frac{\partial f}{\partial U} = \begin{bmatrix} \frac{\partial f_1}{\partial \Delta S} & \frac{\partial f_1}{\partial \Delta \theta} \\ \frac{\partial f_2}{\partial \Delta S} & \frac{\partial f_2}{\partial \Delta \theta} \\ \frac{\partial f_3}{\partial \Delta S} & \frac{\partial f_3}{\partial \Delta \theta} \end{bmatrix} \tag{12}$$

$$B = \begin{bmatrix} \cos(X(3)) & 0 \\ \sin(X(3)) & 0 \\ 0 & 1 \end{bmatrix} \quad \text{OK} \tag{13}$$

2.2 Transformations and Measurement Model

Matrix ${}^oT_m$: Homogeneous transformation from the robot frame m to the world frame o.

$$ {}^oT_m = \begin{bmatrix} \cos(X(3)) & -\sin(X(3)) & X(1) \\ \sin(X(3)) & \cos(X(3)) & X(2) \\ 0 & 0 & 1 \end{bmatrix} \quad \text{OK.} \tag{14} $$

Matrix C: Jacobian of the measurement model with respect to the state X.
Let the real magnet position in the world frame be:

$$ oRealMagnet = \begin{bmatrix} x_m \\ y_m \\ 1 \end{bmatrix} \tag{15} $$

where $\Delta x = x_m - x$ and $\Delta y = y_m - y$.

The expected measurement in the robot frame is:

$$ \hat{Y}_x = \cos(\theta)(x_m - x) + \sin(\theta)(y_m - y) \tag{16} $$

$$ \hat{Y}_y = -\sin(\theta)(x_m - x) + \cos(\theta)(y_m - y) \tag{17} $$

$$ C = \frac{\partial \hat{Y}}{\partial X} = \begin{bmatrix} \frac{\partial \hat{Y}_1}{\partial x} & \frac{\partial \hat{Y}_1}{\partial y} & \frac{\partial \hat{Y}_1}{\partial \theta} \\ \frac{\partial \hat{Y}_2}{\partial x} & \frac{\partial \hat{Y}_2}{\partial y} & \frac{\partial \hat{Y}_2}{\partial \theta} \end{bmatrix} \tag{18} $$

$$ C = \begin{bmatrix} -\cos(X(3)) & -\sin(X(3)) & -\sin(X(3))(oRM_1 - X(1)) + \cos(X(3))(oRM_2 - X(2)) \\ \sin(X(3)) & -\cos(X(3)) & -\sin(X(3))(oRM_2 - X(2)) - \cos(X(3))(oRM_1 - X(1)) \end{bmatrix} \tag{19} $$

OK

2.3 Extended Kalman Filter Equations

Prediction:

$$ P = APA^T + BQ_\beta B^T + Q_\alpha \tag{20} $$

Innovation:

$$ innov = Y - \hat{Y} \tag{21} $$

$$ S = CPC^T + Q_\gamma \quad \text{(Innovation covariance)} \tag{22} $$

Mahalanobis Distance:

$$ d_{\text{Maha}} = \sqrt{innov^T S^{-1} innov} \tag{23} $$

Update:

$$ K = PC^T S^{-1} \quad \text{(Kalman gain)} \tag{24} $$

$$ X = X + K \cdot innov \quad \text{(State update)} \tag{25} $$

$$ P = (I - KC)P \quad \text{(Covariance update)} \tag{26} $$

3 Question 7

3.1 Measurement noise variance estimation

The magnet measurement used by the EKF is:

$$ Y = \begin{bmatrix} \text{sensorPosAlongXm} \\ \text{sensorRes} \cdot (\text{reed\_measurement} - \text{sensorOffset}) \end{bmatrix} \tag{27} $$

So the measurement noise has two components:

$$Q_\gamma = \text{diag}([\sigma_x^2, \sigma_y^2]) \tag{28}$$

The two variances were evaluated separately since they come from different physical effects.

Note: maximum encoder resolution and maximum sampling frequency were used by setting $dumbFactor$ and $subSamplingFactor$ to 1 Relevant only to the determination of sigma_x. But yes.

sigma_x is affected by forward-detection uncertainty: magnet activation width, encoder resolution, and sampling frequency.
sigma_y is affected by lateral quantization: reed spacing / sensor resolution, e.g. 10 mm between reed switches.
Magnet spacing affects association ambiguity, not sigma_x or sigma_y directly.

3.2 Measurement variance on x

The x component of the measurement is assumed to be:

$$Y(1) = 80 \text{ mm} \tag{29}$$

This means the model assumes that the magnet is detected exactly when it is aligned with the reed sensor line. In reality, a reed switch closes when the magnet is inside a finite magnetic activation area. Therefore, the magnet may be detected slightly before or after it is exactly under the sensor. Right.

To estimate this uncertainty, raw reed-switch detections (with diagonal45degrees as input) was used. For each reed sensor, the intervals where the sensor was closed were identified. Then, for each interval, the travelled-distance width was measured using:

$$D = \text{distance at end of detection} - \text{distance at start of detection} \tag{30}$$

This width represents the approximate diameter of the magnetic activation area in the x direction. From the raw data, 28 activation intervals were obtained with a median activation width of 14.26 mm.

Since the sensor only tells us that the magnet was somewhere inside this activation interval, x error was modeled as a uniformly distributed sequence:

Why use the median?
Why not the average?

$$\text{error}_x \sim \text{Uniform}(-D/2, D/2) \tag{31}$$

For a uniform distribution over an interval of width D, the variance is:

$$\sigma_x^2 = \frac{D^2}{12} \tag{32}$$

Using the median width $D = 14.26$ mm gives:

Actually, I disagree with this determination. The only safe way is to make sure you end up with an over-estimation of the noise variance. By taking D as the median rather than the maximum, you do NOT make sure you over-estimate the noise variance. It is potentially dangerous. It may work here, but it is dangerous nonetheless.

$$\sigma_x^2 = \frac{14.26^2}{12} = 16.94 \text{ mm}^2 \tag{33}$$

$$\sigma_x = \sqrt{16.94} = 4.12 \text{ mm} \tag{34}$$

3.3 Measurement variance on y

The y measurement noise has a different origin. It is not caused mainly by the magnetic activation length, but by the physical spacing between reed sensors. The reed sensors are separated by $\text{sensorRes} = 10$ mm. So the lateral position measurement is quantized with a resolution of 10 mm. If one reed switch detects a magnet, the true lateral position could be anywhere within half a sensor spacing on either side of the measured reed-switch position.

Therefore, y error was modeled as:

$$\text{error}_y \sim \text{Uniform}(-\text{sensorRes}/2, \text{sensorRes}/2) \tag{35}$$

Since $\text{sensorRes} = 10$ mm, we have $\text{error}_y \sim \text{Uniform}(-5 \text{ mm}, 5 \text{ mm})$. The variance of a uniform distribution over a width of sensorRes is:

$$\sigma_y^2 = \frac{\text{sensorRes}^2}{12} \tag{36}$$

So:

$$\sigma_y^2 = \frac{10^2}{12} = 8.33 \text{ mm}^2 \tag{37}$$

OK

$$\sigma_y = \frac{10}{\sqrt{12}} = 2.89 \text{ mm} \tag{38}$$

3.4 Measurement covariance

Therefore, the measurement covariance used in the EKF is:

$$\sigma_x = 4.12mm \tag{39}$$
$$\sigma_y = 2.89mm \tag{40}$$
$$Q_\gamma = \text{diag}([\sigma_x^2, \sigma_y^2]) \tag{41}$$

Replacing:

$$Q_\gamma = \text{diag}([16.94, 8.33]) \tag{42}$$

4 Question 8

The Mahalanobis distance is used to decide whether a magnet measurement is compatible with the predicted measurement. This is important because the magnets are unsigned: when the robot detects a magnet, it does not directly know which magnet it detected. Therefore, after associating the detection with the closest magnet, the Mahalanobis distance is used to check whether this association is statistically reasonable. We had a similar idea in an example in class with unsigned beacons.

The squared Mahalanobis distance is:

$$d^2 = Z^T S^{-1} Z \tag{43}$$

A large value indicates that the difference between the real measurement and the predicted measurement is large compared with the expected uncertainty (bad). However, how can we determine if a value is ”too large”.

If the innovation Z is a zero-mean Gaussian random vector, then the squared Mahalanobis distance follows a chi-square distribution:

$$d^2 \sim \chi^2(\dim(Z)) \tag{44}$$

In this lab, the measurement vector has two components:

$$Y = \begin{bmatrix} x_{\text{meas}} \\ y_{\text{meas}} \end{bmatrix} \tag{45}$$

Therefore, $\dim(Z) = 2$, so the squared Mahalanobis distance follows a chi-square distribution with 2 degrees of freedom.

To choose the threshold, a confidence probability of $p = 0.95$ was used. Meaning that 95% of correct measurements should have d^2 smaller than the threshold (5.99 in this case).

Moreover, if the probability of the innovation is less than 5%, the measurement is considered abnormal. In this case, the most likely explanation is that the measurement was associated with the wrong magnet, so it is rejected.

Note: The code uses d not d^2 it is the same idea just everything will have a $sqrt$ function. OK

5 Question 9

5.1 Methodology for tuning `sigmaTuning`

After estimating the measurement noise covariance Q_γ, choosing the Mahalanobis threshold, and setting the initial covariance matrix P_{init}, the remaining main tuning parameter is `sigmaTuning`.

For this tuning step, the robot parameters were reset to the normal values used for the Kalman filter:

$$dumbFactor = 8 \tag{46}$$

$$subSamplingFactor = 4 \tag{47}$$

This means that the encoder resolution is artificially degraded and the data is processed at 5 Hz (As stated at the start of the given).

The initial covariance matrix was chosen from the estimated uncertainty when manually placing the robot at its initial posture:

$$\sigma_X = 5 \text{ mm} \tag{48}$$
$$\sigma_Y = 5 \text{ mm} \tag{49}$$
$$\sigma_\theta = 5^\circ = \frac{5\pi}{180} \text{ rad} \tag{50}$$

It's a reasonable order of magnitude

Therefore:

$$P_{init} = \begin{bmatrix} \sigma_X^2 & 0 & 0 \\ 0 & \sigma_Y^2 & 0 \\ 0 & 0 & \sigma_\theta^2 \end{bmatrix} \tag{51}$$

5.2 Role of `sigmaTuning`

The parameter `sigmaTuning` represents the uncertainty on the wheel encoder increments. In the code, it is used to define: I disagree, here. Because Qalpha is set to zero, Qbeta (and hence sigma_tuning) represents ALL the sources of errors of the odometry process, not just the uncertainty on the measurement of wheel rotation.

$$Q_{wheels} = \sigma_{tuning}^2 I_2 \tag{52}$$

Then this uncertainty is transformed from wheel coordinates into robot motion coordinates:

$$Q_\beta = jointToCartesian \; Q_{wheels} \; jointToCartesian^T \tag{53}$$

The covariance prediction step is:

$$P = APA^T + BQ_\beta B^T + Q_\alpha \tag{54}$$

In this lab, $Q_\alpha = 0$, so the main source of covariance growth during prediction is:

$$BQ_\beta B^T \tag{55}$$

Therefore, `sigmaTuning` controls how much uncertainty is added when the robot moves using odometry.

If `sigmaTuning` is too small, the filter trusts odometry too much. The covariance P grows too slowly, and therefore the innovation covariance

$$S = CPC^T + Q_\gamma \tag{56}$$

also remains too small. Since the Mahalanobis distance is computed as

$$d_{Maha} = \sqrt{innov^T S^{-1} innov} \tag{57}$$

a small S gives a large S^{-1}, so even correct magnet measurements can produce a large Mahalanobis distance and be rejected.

If `sigmaTuning` is too large, the covariance P grows too quickly. Then $CPC^T + Q_\gamma$ becomes large, which makes the Mahalanobis gate too permissive. In this case, wrong magnet associations may be accepted, and the estimated trajectory may become jumpy or unstable.

Okay, you understand the "mechanics" of the equations and of the influence of the choice of sigmaTuning.

5.3 Tuning experiment

The main tuning was performed using the `twoloops` dataset. The robot is expected to finish close to the starting point, so the final EKF position can be compared with the expected final position near the origin (this realization during tuning helped me better evaluate the parameters).

A very small value was first tested:

$$\sigma_{tuning} = 0.01 \tag{58}$$

For the `twoloops` dataset, the results were:

```
Final odometry error: 0.5 %
Magnets rejected: 73.8 %
Neighbor magnets under threshold: 0.0 %

% Final EKF position: x = 93.17 mm, y = -24.97 mm
% Final EKF distance to origin: 96.46 mm
```

Initially, my mistake was basing my judgment solely on minimizing the final odometry error. While increasing σ_{tuning}, I noticed that the final odometry error increased whereas the rejected magnets decreased. After some analysis, I found that relying on a small final odometry error is misleading. The EKF rejected 73.8% of the magnet measurements, causing it to behave almost like pure odometry, which explains the small error obtained. Here, the error measures the difference between the final odometry-only position and the final EKF position, normalized by the traveled distance. The whole point of hybrid localization is for the EKF to correct the odometry when a reliable magnet measurement is available. To overcome this, I leveraged the fact that since the trajectory is a loop, the robot must return to its initial position. Therefore, I added the final EKF position and the final EKF distance to the origin as evaluation metrics.

Yes!

Yes

After multiple iterations, a better value was obtained with:

$$\sigma_{tuning} = 0.08 \tag{59}$$

For the `twoloops` dataset, the results were:

```
Final odometry error: 4.6 %
Magnets rejected: 3.7 %
Neighbor magnets under threshold: 0.0 %

Final EKF position: x = 2.95 mm, y = 0.38 mm
Final EKF distance to origin: 2.97 mm
```

Neighbor magnets fall under the Mahalanobis threshold when the uncertainty gate is too large compared with magnet spacing. This can happen if Pinit or Q is over-estimated, if the Mahalanobis threshold is too permissive, if odometry uncertainty grows too much between updates, if the initial pose is ambiguous, or if the magnets are physically too close.

This result is much better. The final EKF position is very close to the expected final position near the origin, and only 3.7% of magnet measurements were rejected. Additionally, the percentage of neighboring magnets under the Mahalanobis threshold remained 0.0%, which means the filter did not become too permissive with respect to wrong magnet associations. Yes. The percentage MUST be 0%.

The final odometry error is larger than in the poorly tuned case, but this is not a problem. It means that the EKF estimate has moved away from the odometry-only estimate because magnet corrections were successfully applied. Yes

I also tested it on other datasets and obtained satisfactory results.

6 Question 10

Starting from the correctly tuned filter, each parameter was modified separately. The goal was to understand how the filter behavior changes, especially through the innovation covariance:

$$S = CPC^T + Q_\gamma \tag{60}$$

This term affects both the Mahalanobis distance and the Kalman gain.

Physical magnet detection is fixed by the hardware. The Mahalanobis gate is only a software test used after detection to decide which known map magnet is compatible with the measurement. A larger S does not change the sensor; it only makes the software acceptance region larger, so more candidate magnets may look plausible.

6.1 a) Initial robot position variance

The initial covariance P_{init} represents the uncertainty in the initial robot posture.

When P_{init} was under-estimated, the filter assumed that the initial position was almost perfectly known. Therefore, P started too small, so S was also too small at the beginning. This made the Mahalanobis distance larger and made the filter more likely to reject valid magnet measurements. The EKF then behaved closer to odometry at the start. The problem is solved because P increases since measurements are rejected.

When P_{init} was over-estimated by multiplying the standard deviations by 10, P started very large. Therefore, S became large, making the Mahalanobis gate too permissive. Measurements were accepted more easily, but this can be dangerous because neighboring magnets may also become acceptable. The estimated standard deviations started very large and then decreased sharply after magnet updates.

Here it is interesting to note that if Pinit is overestimated but not too much, it makes no difference after a few measurements have been taken into account, and no magnet identification ambiguity exists (red dots below the threshold).

6.2 b) Measurement noise variance

The measurement noise covariance Q_γ represents the uncertainty of the magnet measurement.

When Q_γ was under-estimated by setting the measurement standard deviations to zero, the filter assumed that the magnet sensor was perfect. Accepted measurements made the covariance shrink too much, sometimes almost to zero. Then $CPC^T + Q_\gamma$ became too small, which increased the Mahalanobis distance and caused many later measurements to be rejected.

In the test, this gave: The important point here is that the robot is lost: it no longer maintains a correct localization.

```
Magnets rejected: 64.4 %
Final EKF distance to origin: 28.29 mm
```

When Q_γ was over-estimated by multiplying the measurement standard deviations by 10, $CPC^T + Q_\gamma$ became very large. This made the Mahalanobis gate too permissive. Almost all measurements were accepted, but neighboring magnets also passed the threshold. All, I would say.

In the test, this gave:

```
Magnets rejected: 0.0 % Indeed, all. Makes sense.
Neighbor magnets under threshold: 100.0 %
```

So over-estimating Q_γ makes the filter unable to clearly distinguish the correct magnet from neighboring magnets. Of course such an error can only occur when, say you use the wrong units...

6.3 c) Tuning parameter `sigmaTuning`

Seen in question 9.

If Pinit is too large, the software acceptance region becomes large compared to the 55 mm spacing between magnets, so neighbor magnets can fall under the threshold.

6.4 d) Slight over-estimation of initial position variance

A slight over-estimation of P_{init} is usually acceptable. It makes the filter a little less confident at the beginning, so early magnet measurements are easier to accept.

For the slight over-estimation test of P_{init}, several initial standard deviations were tested: 5, 10, 20, 30, 40, 50, 100, 200 and 500. On the `oneloop` dataset, the final EKF position remained almost unchanged. However, the first sign of a problem appeared at approximately $\sigma_X = \sigma_Y = 20$ mm and $\sigma_\theta = 20°$, where neighboring magnets started to fall under the Mahalanobis threshold. Normal considering inter-magnet distances, right?

For 5 mm and 10 mm, the percentage of neighboring magnets under threshold was 0.0%. At 20 mm it became 1.0%, and for larger values it stayed around 1.4%. This means that the filter became too uncertain at the beginning, making $CPC^T + Q_\gamma$ too large and the Mahalanobis gate too wide.

Therefore, for this dataset, the practical limit is around 20 mm / 20 degrees. Beyond this value, the final position may still look correct, but the data association becomes less reliable because neighboring magnets can also satisfy the Mahalanobis test. Anything other than 0% of ambiguity is unacceptable.

I felt like the limit is maybe even higher, as I am not sure these numbers are high enough to consider it as truly unacceptable. You have the correct grasp. The important is what it means: it is not necessary to accurately know Pinit. Over-estimation is okay, as long as matching measurements to a single magnet is still possible. The way to realize that is to compare the evolution of sigma_x, sigma_y, sigma_theta over time for two values of Pinit, and see that after a few measurements, there is no noticeable difference.

7 Question 11

7.1 b) Sharp correction on the `twoloops` trajectory

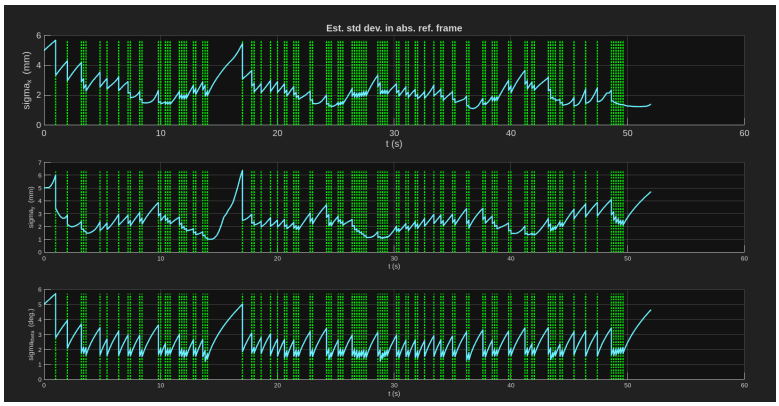
On the `twoloops` trajectory, a fairly sharp correction can appear around the point $(60, 260)$. This correction is not necessarily an error in the EKF. It can be explained by the evolution of the covariance and the availability of magnet measurements.

Before this correction, the robot goes through a part of the trajectory where the filter relies mainly on odometry, either because fewer magnets are detected or because fewer measurements are accepted. During this interval, odometry errors accumulate and the uncertainty increases through the prediction step. This can also be seen in the standard deviation plots, where the estimated uncertainties increase during intervals with fewer magnet corrections. only (3 s)

When magnet measurements become available again, the innovation may be relatively large because the odometry estimate has drifted. However, since P has also increased, the innovation covariance is larger. Therefore, the Mahalanobis distance may still remain below the threshold, so the measurement is accepted.

The EKF then applies the correction, because the accumulated drift and the covariance are both significant, this update can be visually sharp on the estimated trajectory.

Therefore, the correction around $(60, 260)$ is mainly caused by accumulated odometry drift followed by a valid magnet correction. It shows the EKF recovering when landmark information becomes available again.



7.2 c) Two diverging straight lines

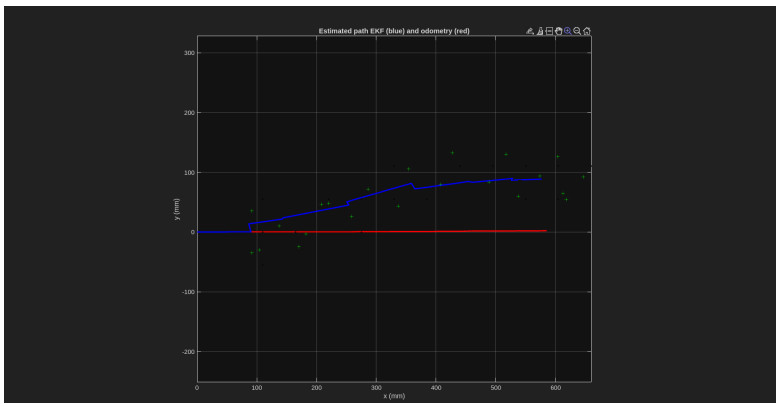
In the `diagonal45degrees` dataset, the expected trajectory is an approximately straight path at $45°$. Ideally, the odometry and EKF estimates should therefore follow nearly the same diagonal direction. However, the result shows two almost straight trajectories with different slopes.

This behavior can be explained by a small systematic heading error in the odometry. Although the robot was physically driven along an approximately diagonal path, a slight mismatch between the true orientation of the robot and the orientation estimated from the wheel encoders caused the odometry to propagate in a slightly incorrect direction. As the robot continued moving forward, this small angular bias accumulated and produced an increasing lateral deviation, as expected from the motion model.

The magnetic measurements help reduce this drift by periodically correcting the estimated pose, so the EKF trajectory remains closer to the real path. Since the robot motion is still dominated by one main direction, both estimates appear almost linear, but they do not remain aligned because they are based on different heading corrections.

Yes, there is no possibility to correct theta in the odometry process. If delta_theta is measured as zero, thetha stays constant (and incorrect).

But now that I see your figure below, I realize you give the correct explanation for the wrong reasons. You have run the filter with an initial orientation of 0. Nothing makes sense in your results! Neither odometry nor the EKF is correct. You probably have a lot of rejections, here.



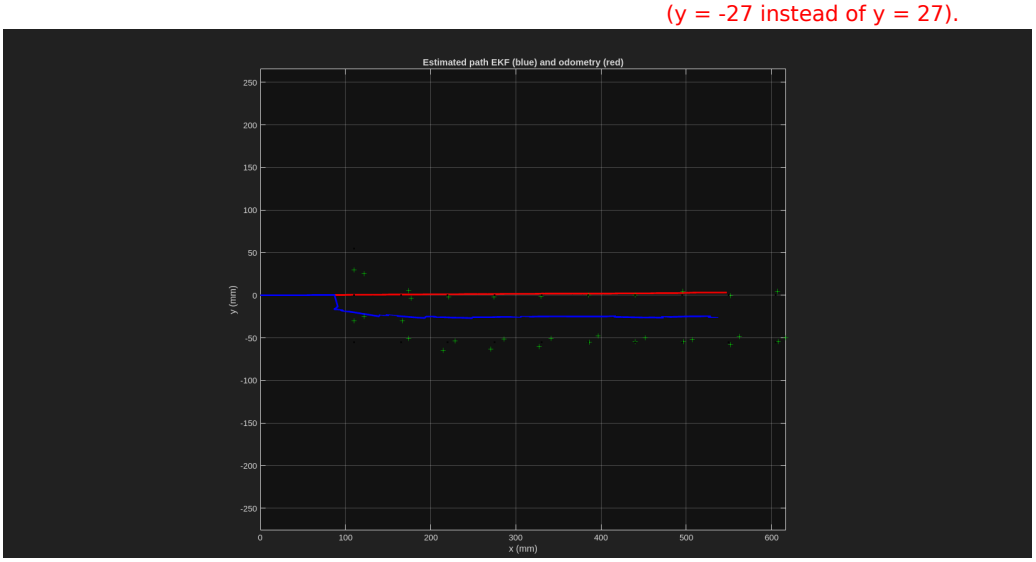
7.3 d) Effect of initialization on `line2magnets`

For the `line2magnets` dataset, the correct initial posture is $(0, 27, 0)$, not $(0, 0, 0)$. This is important because the robot moves along a line located between two rows of magnets.

When the filter was incorrectly initialized at $(0, 0, 0)$, the EKF associated the measurements with the wrong row of magnets. The final estimate was:

```
Final EKF position: x = 536.82 mm, y = -25.81 mm
Magnets rejected: 34.4 %
Neighbor magnets under threshold: 0.8 %
```

The problem actually is that, initially, Mahalanobis distances are above the threshold because of an unreasonably large initial error (in y).
When P increases because of measurement rejections, measurements are eventually accepted, but with a matching to the incorrect row of magnets (y = -27 instead of y = 27).



So the trajectory remained smooth, but it was localized around the wrong lateral position.

When the filter was correctly initialized at $(0, 27, 0)$, the EKF associated the detections with the correct magnet row. The final estimate was:

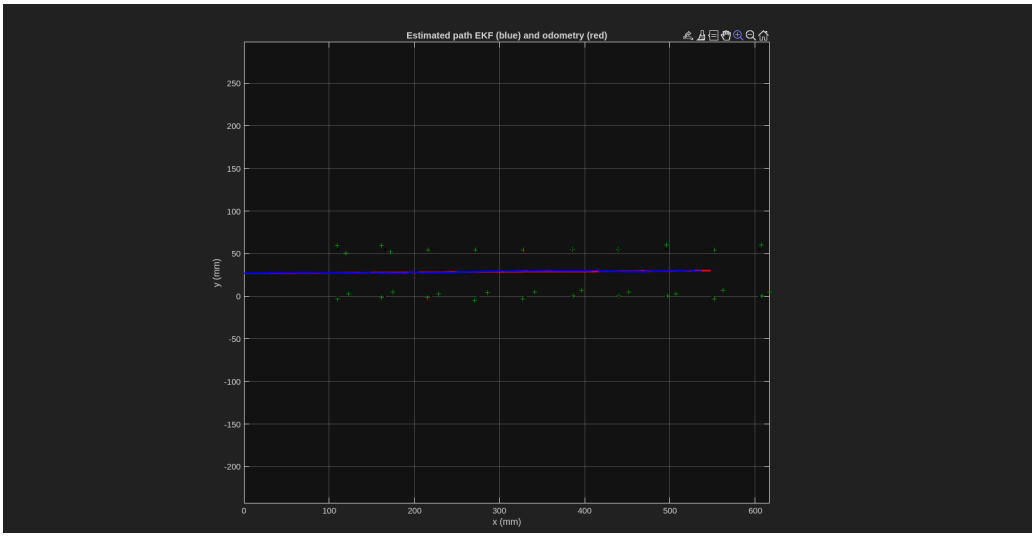
```
Final EKF position: x = 536.81 mm, y = 29.18 mm
Magnets rejected: 18.8 %
Neighbor magnets under threshold: 0.0 %
```

This is related to the fact that the state is LOCALLY observable. Clearly, the neighborhood within which the initial state is know has to be smaller in radius than the half-distance between two magnets.
Here is iniial error is equal to the half-distance, making the task of determining the state impossible.

This shows that the EKF is sensitive to the initial posture because the magnets are unsigned. The sensor detects a magnet but does not know its identity. Therefore, the algorithm must associate each detection with a magnet from the grid. If the initial pose is wrong, the nearest grid magnet can belong to the wrong row, and the filter may consistently localize on that wrong row. Yes

The Mahalanobis test can reject unlikely measurements, but it cannot always correct a wrong association if the wrong magnet is still statistically plausible. Therefore, correct initialization is essential for this dataset.

Good initialization is not optional. With unsigned magnets, the initial pose must be known accurately enough that only one magnet/row is plausible. If the initial error reaches half the spacing between possible magnets or rows, the filter may lock onto the wrong association.



7.4 e) Comparison of estimated standard deviations

In Figure 3 (robot frame), the estimated standard deviations for x and y are different because the robot moves mainly along the x direction while the magnetic information primarily constrains the lateral (y) position.

Along x, uncertainty grows more between updates because forward motion continuously integrates odometry errors. The magnet gives only limited correction in this direction, so σ_x increases more noticeably.

Along y, the single magnet line provides strong lateral reference: when a magnet is detected, the robot can accurately correct its sideways offset relative to that line. Therefore σ_y is reduced more strongly and remains smaller.

Not a correct expression. It's observable or not.

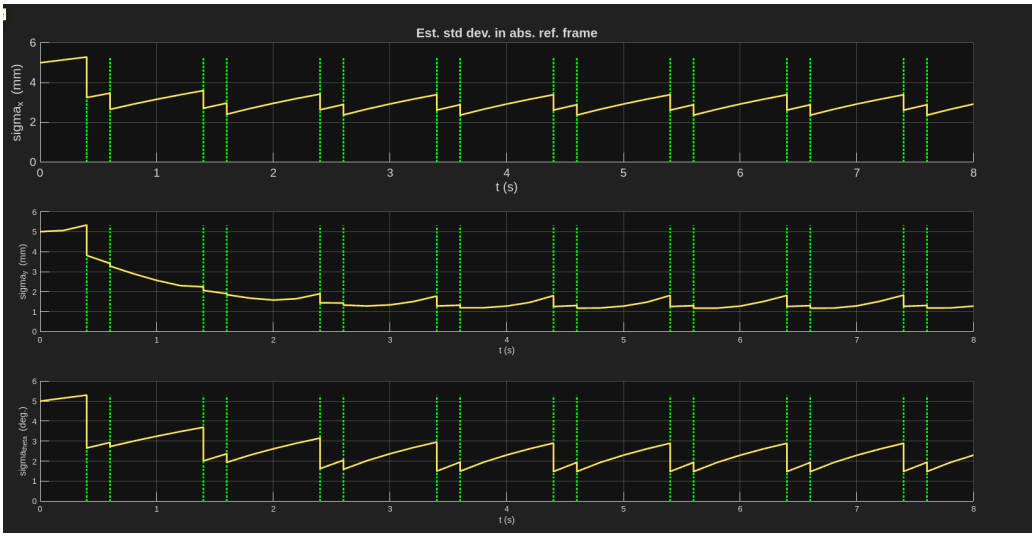
We can conclude that, motion direction (x) is ~~less observable~~ than lateral direction (y).

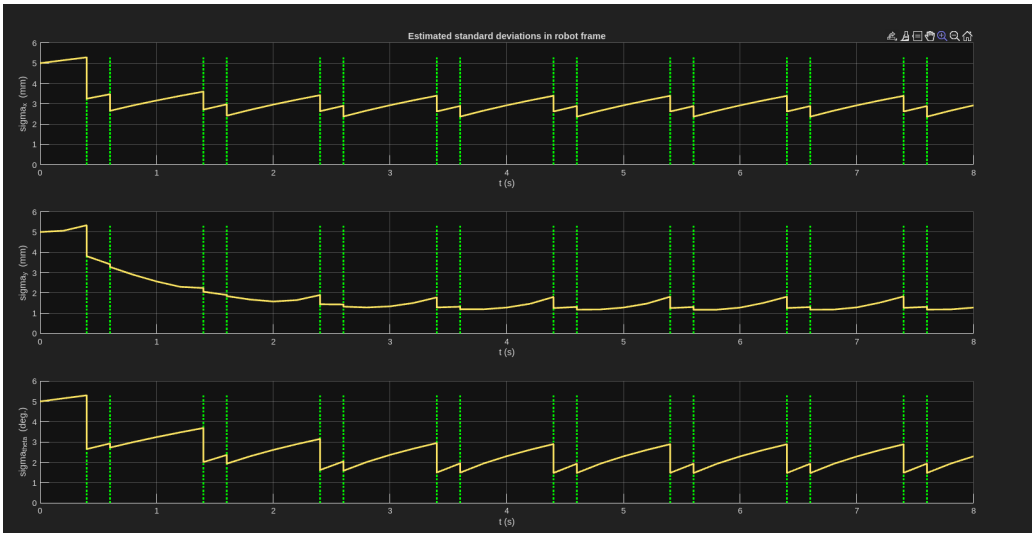
In Figures 3 and 4, the curves are almost the same because the robot orientation is close to $0°$ for the whole trajectory (it moves essentially straight). Since the robot frame and the absolute reference frame are nearly aligned, transforming the covariance from robot coordinates to global coordinates produces almost no change (if $\theta \approx 0$, the rotation matrix is close to identity).

yes

If the robot were turning significantly, Figures 3 and 4 would differ because the uncertainty ellipse would rotate with the robot frame (seen below).

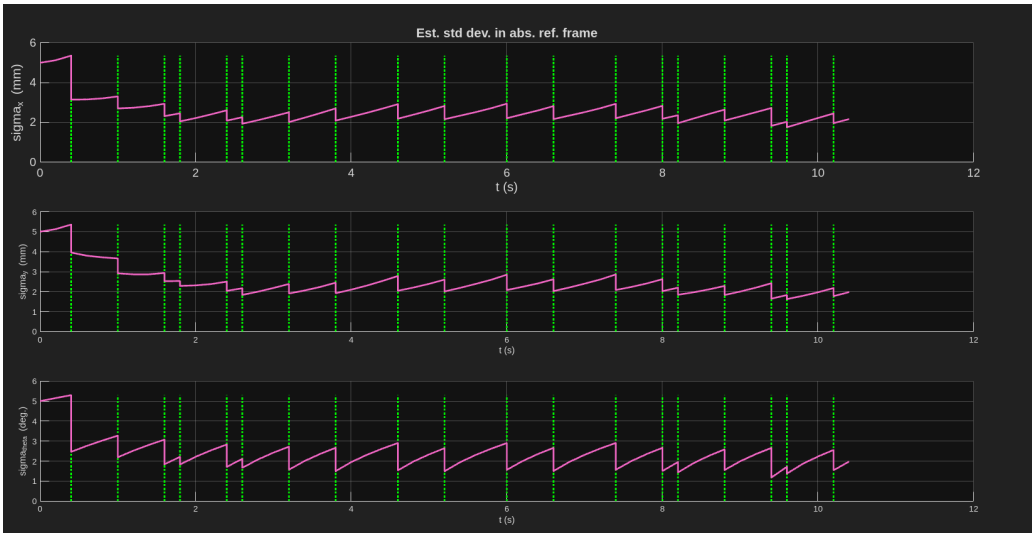
yes





7.5 f) Standard deviations for `diagonal45degrees`

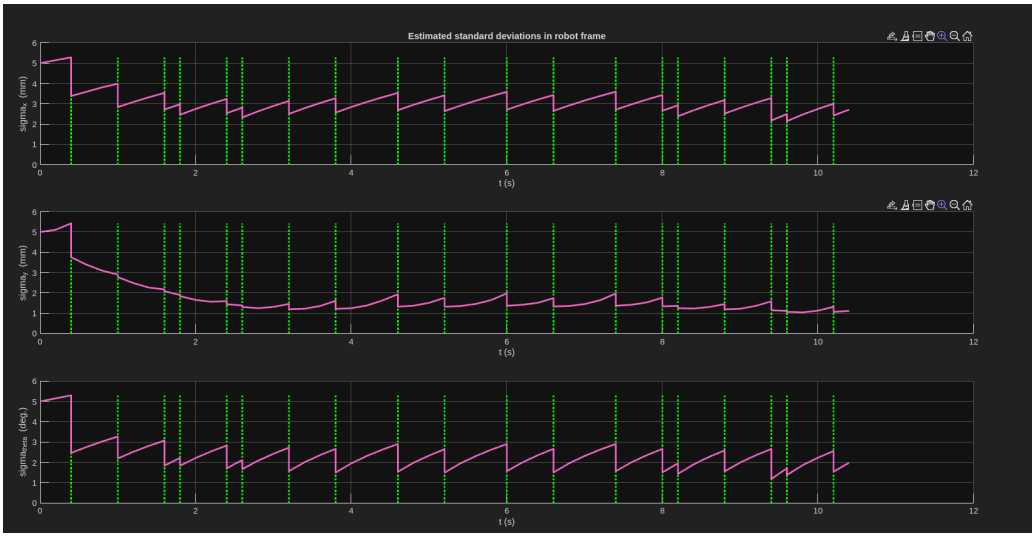
For the `diagonal45degrees` dataset, in Figure 3, the estimated standard deviations for the x and y components in the absolute frame are almost the same.



This happens because the robot moves approximately at $45°$ with respect to the world frame. The main uncertainty is aligned with the robot motion, but since this motion direction is diagonal, it is projected almost equally onto the world x and y axes. Therefore, the absolute-frame uncertainties in x and y look similar.

Figures 3 and 4 are different because they express the same covariance in two different frames. Figure 3 uses the absolute/world frame, while Figure 4 uses the robot frame.

In the robot frame, the uncertainty is expressed along the robot's own axes. These directions are physically meaningful because odometry and magnet corrections do not affect them in the same way. The robot moves mainly forward, so longitudinal uncertainty and lateral uncertainty can be different.



7.6 g) Circles dataset

In the `circles` dataset, the EKF path can look unusual or as if it is drifting. However, this does not necessarily mean that the EKF is wrong or drifting out of control.

The robot is continuously turning, so its local frame rotates with respect to the world frame. Because of this, the uncertainty projected onto the absolute x and y axes changes continuously. Therefore, the world-frame standard deviation plots can be misleading.

The robot-frame standard deviations are more meaningful here, because they express uncertainty along the robot's forward and lateral directions. If these uncertainties remain bounded and decrease when magnet measurements are accepted, then the EKF is behaving correctly.

Therefore, the circles dataset mainly shows why plotting uncertainty in the robot frame is useful. The EKF may look strange in the absolute frame, but it is not necessarily diverging. As long as the covariance remains bounded and magnet updates reduce uncertainty, the filter is not drifting out of control.

Actually, the idea of this question was not related to comparing sigma_x and sigma_y in robot/absolute frame.
You did that well above, and it looks like you understand correctly.
The idea was to make up your mind on whether the EKF is correct or not. The way to justify that is to look at the Mahalanobis distances. Each time a magnet is detected, it is successfully matched to at most one (generally exactly one) magnet. That proves that everything works as it should.